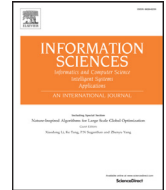




Contents lists available at ScienceDirect

## Information Sciences

journal homepage: [www.elsevier.com/locate/ins](http://www.elsevier.com/locate/ins)

# An evolving approach to data streams clustering based on typicality and eccentricity data analytics



Clauber Gomes Bezerra<sup>a,\*</sup>, Bruno Sielly Jales Costa<sup>b</sup>, Luiz Affonso Guedes<sup>c</sup>, Plamen Parvanov Angelov<sup>d</sup>

<sup>a</sup> Federal Institute of Education, Science and Technology of Rio Grande do Norte - IFRN, Campus Natal Zona Leste Av. Senador Salgado Filho 1559, Tirol, CEP: 59015-000, Natal, RN, Brazil

<sup>b</sup> Federal Institute of Education, Science and Technology of Rio Grande do Norte - IFRN, Campus Natal Zona Norte Rua Brusque 2926, Potengi, CEP 59112-490, Natal, RN, Brazil

<sup>c</sup> Federal University of Rio Grande do Norte - UFRN, Department of Computer Engineering and Automation - DCA Campus Universitário, Lagoa Nova, CEP: 59078-900, Natal, RN, Brazil

<sup>d</sup> Lancaster University, Data Science Group, School of Computing and Communications, Lancaster LA1 4WA, United Kingdom

## ARTICLE INFO

### Article history:

Received 24 May 2019

Revised 13 December 2019

Accepted 15 December 2019

Available online 2 January 2020

### MSC:

00-01

99-00

### Keywords:

Online clustering

Data stream

Eccentricity

Typicality

Anomaly detection

## ABSTRACT

In this paper we propose an algorithm for online clustering of data streams. This algorithm is called AutoCloud and is based on the recently introduced concept of Typicality and Eccentricity Data Analytics, mainly used for anomaly detection tasks. AutoCloud is an evolving, online and recursive technique that does not need training or prior knowledge about the data set. Thus, AutoCloud is fully online, requiring no offline processing. It allows creation and merging of clusters autonomously as new data observations become available. The clusters created by AutoCloud are called data clouds, which are structures without pre-defined shape or boundaries. AutoCloud allows each data sample to belong to multiple data clouds simultaneously using fuzzy concepts. AutoCloud is also able to handle concept drift and concept evolution, which are problems that are inherent in data streams in general. Since the algorithm is recursive and online, it is suitable for applications that require a real-time response. We validate our proposal with applications to multiple well known data sets in the literature.

© 2020 Elsevier Inc. All rights reserved.

## 1. Introduction

Data clustering is the basis for solving several different classes of problems in various fields of application such as data mining, pattern recognition, data classification, system identification and image segmentation [1–3]. Due to the great demand for it, the task of clustering data has been the object of study of many different authors over the past decades, resulting in a great variety of approaches presented in the literature. The intrinsic mechanisms of each of these approaches define its realm of applicability.

Most of the approaches proposed in the literature require some prior knowledge about the analyzed data set [1]. This knowledge is often presented in the form of a mathematical model that describes the behaviour of the data. In other cases,

\* Corresponding author.

E-mail addresses: [clauber.bezerra@ifrn.edu.br](mailto:clauber.bezerra@ifrn.edu.br) (C.G. Bezerra), [bruno.costa@ifrn.edu.br](mailto:bruno.costa@ifrn.edu.br) (B.S.J. Costa), [affonso@dca.ufrn.br](mailto:affonso@dca.ufrn.br) (L.A. Guedes), [p.angelov@lancaster.ac.uk](mailto:p.angelov@lancaster.ac.uk) (P.P. Angelov).

prior training is required, using a representative set of data, allowing the algorithm to learn some of the patterns in this set. For instance, traditional clustering techniques, such as k-means [4], require the offline availability of the whole data set from the start. However, there are several problems in which the data samples are acquired over time, during the execution of the algorithm, such as general data streams.

A data stream is an ordered sequence of samples, obtained continuously over time, such as a time series. Since the order of the samples is a crucial feature of the problem and they are usually obtained in an online manner, this type of data set requires not only an algorithm that can perform clustering online, but also has the ability to adapt, since very little knowledge about the data is available *a priori* [5]. Thus, the algorithm must be able to handle, during the analysis of the data stream, the occurrence of problems such as concept evolution and concept drift.

Algorithms and techniques designed to handle this and other problems that involve data sets in dynamic and non-stationary environments are often called evolving intelligent systems [6]. One of the main features of such systems is their ability to adapt and evolve autonomously according to the natural changes in the data over time. Evolving intelligent systems have been the object of study of many authors [7,8] and solutions based on this concept have been recently introduced for many different problems [9], such as systems modeling, process controls, data prediction and classification [10–16].

Recently, several incremental learning algorithms have been proposed in the context of evolving clustering. Many of them assume a specific format for clusters, such as ellipsoids. It is important to note that ellipsoids, for example, assume that the data follow a Gaussian distribution. Lughofer and Sayed-Mouchaweh [17] and Lughofer [18] are two great examples of successful attempts to tackle evolving clustering that are based on this assumption. In [17], the authors propose an interesting approach based on the evolving vector quantization method that was originally published in [18], named AutoClust. The proposed algorithm has merge-and-split functionalities for the clusters. It assumes that the clusters have ellipsoid shapes, and then it uses the geometric information about the overlaps of the clusters and joint cluster homogeneity to merge and/or split them. In [19], the authors present a fuzzy evolving clustering approach that also assumes ellipsoidal shape for the clusters, using a Gaussian mixture model and fuzzy inference to define the clusters and their evolution.

In this paper we propose an algorithm for data stream clustering entitled AutoCloud. AutoCloud is fully data driven and does not require specific mathematical models or any prior knowledge about the data set to be analyzed. It is based on the recently introduced concept of Typicality and Eccentricity Data Analytics (TEDA), mostly applied to anomaly detection problems. Although in the current form of AutoCloud its eccentricity calculation is based on Euclidean distance, which virtually forces the definition of clusters as ellipses, the underlying idea of the algorithm does not assume any pre-defined shape, as other similarity measures can be used and data clouds, by definition, do not have specific shapes or boundaries [20]. Therefore, AutoCloud is able to perform evolving clustering for data streams fully online and very low computational complexity.

Algorithms for evolving clustering in arbitrarily shaped clusters have been introduced and follow the classic DBSCAN algorithm [21], with concepts of micro and macro clusters. These algorithms work generally in two stages: micro-clustering is performed online while macro-clustering is executed offline, as in [22–24]. From another perspective, Hyde et al. [25] proposes a fully online clustering of evolving data streams with arbitrarily shaped clusters, named CEDAS, that updates the micro-clusters and macro-clusters online. It uses a graph structure to associate a set of micro-clusters to a macro-cluster, where the micro-clusters are the nodes and the edges are their pairs with intersecting micro-clusters. More recently, Islam [26] proposed the BOCEDS algorithm, which is also a fully online density-based clustering algorithm using the concepts of micro-clusters and macro-clusters for evolving data streams. It uses an energy function based on the spatial information of the data stream for online updating of the micro-clusters. In addition, it adopts a buffer for storing temporarily the micro-clusters, and when these micro-clusters are non-significant, they are removed from the buffer. Two-stage algorithms are hard to compare with the approach presented in this paper, since we decided to focus on a fully online solution. If a scenario permits a training stage, AutoCloud can be easily adapted to perform micro/macro clustering.

AutoCloud is also an evolving algorithm, being able to autonomously adapt to changes in the data over time, such as concept drift and concept evolution [6]. It is also recursive and online, which means it does not require storing and processing past data samples. This greatly reduces the computational burden, computer- and memory-wise. One of the main practical advantages of AutoCloud is that it introduces an elegant and efficient way of creating new clusters and merging existing ones according to the evolution of data over time.

The remainder of this paper is organized as follows. In Section 2 we present the concepts of TEDA. In Section 3, we detail our proposed technique. In Section 4, we describe and discuss the experiments and results obtained with the application of AutoCloud to several well-known data sets. Finally, in Section 5, we present our conclusions and discuss potential future research.

## 2. Typicality and eccentricity data analytics

TEDA is an evolving method for the detection of anomalies and outliers, introduced by Angelov [20]. However, the concepts of typicality and eccentricity have been successfully applied to, among other problems, data classification [27,28]. The concept of typicality is related to the similarity of a specific  $n$ -dimensional data sample to the values of its past readings. Eccentricity, in contrast, describes how different a data sample is from the data distribution. Hence, a data sample with high eccentricity (and thus low typicality) is usually an anomaly.

Eccentricity is summarized as the sum of the distances of a particular data sample to all other existing data samples, divided by the sums of the distances from all data samples to all other data samples. In the realm of data streams, consider a data input  $X \in \mathbb{R}^n$ , which consists of a sequence of  $n$ -dimensional data samples, i.e.  $X = \{x_1, x_2, \dots, x_k, \dots\}$ ,  $x_i \in \mathbb{R}^n$ ,  $i \in \mathbb{N}$ , where  $k$  is the discrete time instant in which the sample was acquired. Consider also  $d(x_i, x_j)$  as a distance function between samples  $x_i$  and  $x_j$ , in which  $d$  can be any type of distance function such as Euclidean, cosine or Mahalanobis. For the complete

$$\pi_k(x) = \sum_{i=1}^k d(x, x_i) \quad (1)$$

where  $\pi_k(x)$  is the sum of the distances from a particular sample  $x \in X$ , to each of the  $k$  elements of the data set.

The eccentricity  $\xi$  of the data sample  $x$  at the time instant  $k$  is defined as [20]

$$\xi_k(x) = \frac{2\pi_k(x)}{\sum_{i=1}^k \pi_k(x_i)} = 2 \frac{\sum_{i=1}^k d(x, x_i)}{\sum_{i=1}^k \sum_{j=1}^k d(x_i, x_j)}, \quad k \geq 2, \quad \sum_{i=1}^k \pi_k(x) > 0 \quad (2)$$

However, Eq. (2) is an offline formula for the eccentricity. It has been shown [20] that the eccentricity can be derived exactly as

$$\xi_k(x) = \frac{1}{k} + \frac{(\mu_k^x - x_k)^T (\mu_k^x - x_k)}{k[\sigma^2]_k^x}, \quad [\sigma^2]_k^x > 0 \quad (3)$$

where  $\xi_k(x)$  is the eccentricity of the sample  $x_k$  in relation to all previous samples in the data set, while  $\mu_k^x$  is the mean and  $[\sigma^2]_k^x$  is the aggregated variance of  $x$  up to the time instant  $k$ . Both  $\mu_k^x$  and  $[\sigma^2]_k^x$  can be recursively updated by

$$\mu_k^x = \frac{(k-1)}{k} \mu_{k-1}^x + \frac{1}{k} x_k, \quad k \geq 1, \quad \mu_0^x = 0 \quad (4)$$

$$[\sigma^2]_k^x = \frac{(k-1)}{k} [\sigma^2]_{k-1}^x + \frac{1}{k} \|x_k - \mu_k^x\|^2, \quad k \geq 1, \quad [\sigma^2]_0^x = 0 \quad (5)$$

Conversely, the  $\tau$  of the sample  $x$  at the time instant  $k$  can be calculated as a complement of the eccentricity, as in [20]:

$$\tau_k(x) = 1 - \xi_k(x) \quad (6)$$

In Section 3, we will develop the idea of how eccentricity in its form presented in Eq. (3) can be used to determine whether a data point belongs to an existing data cloud and how it can be used to determine when a new data cloud must be created to compensate for a potential data drift. The eccentricity and typicality are bounded by Angelov [20]

$$\begin{aligned} 0 \leq \xi_k(x) \leq 1, \quad \sum_{i=1}^k \xi_k(x_i) &= 2, \quad k \geq 2 \\ 0 \leq \tau_k(x) \leq 1, \quad \sum_{i=1}^k \tau_k(x_i) &= k - 2, \quad k \geq 2 \\ \sum_{i=1}^k \pi_k(x_i) &> 0, \quad k \geq 2 \end{aligned}$$

Finally, the normalized eccentricity  $\zeta_k(x)$  and typicality  $t_k(x)$  can be obtained by Angelov [20]

$$\zeta_k(x) = \frac{\xi_k(x)}{2}, \quad \sum_{i=1}^k \zeta_i(x) = 1, \quad k \geq 2 \quad (7)$$

$$t_k(x) = \frac{\tau_k(x)}{k-2}, \quad \sum_{i=1}^k t_i(x) = 1, \quad k \geq 2 \quad (8)$$

The threshold used to distinguish normal from anomalous data samples is based on the Chebyshev inequality [29], which states that, under any distribution, no more than  $1/m^2$  of the data observations are more than  $m\sigma$  away from the mean, where  $\sigma$  is the standard deviation of the data. Thus, a particular data sample  $x_k$  is considered to be an anomaly if the condition

$$\zeta_k > \frac{m^2 + 1}{2k}, \quad m > 0 \quad (9)$$

is satisfied. The parameter  $m$  is user defined and directly affects the sensitivity of the anomaly detector. Although it can be defined using multiple criteria,  $m = 3$  is largely used in literature [8,30] as a standard value and leads to satisfactory results

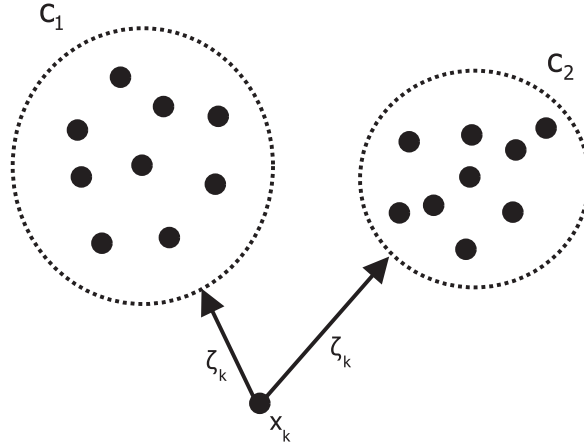


Fig. 1. Overview of AutoCloud.

for different data sets and different configurations. Once again, Eq. (9) is central to the method proposed in this paper, and in Section 3, we will demonstrate how it is used for the determination of the membership of points in an existing data cloud and potentially triggering the creation of new ones.

### 3. AutoCloud

The evolving clustering algorithm proposed in this paper is called AutoCloud. It is based on TEDA, which makes it suitable for the online processing of data streams. Among its characteristics, one can mention:

- (i) It leverages ever-evolving cluster-like granular structures - the data clouds - where not only the parameters of each granule can be adapted, but also new clusters can be created and existing ones merged, which makes it suitable for handling dynamic and evolving data (concept drift and concept evolution).
- (ii) The calculations are recursive, so that it does not require storing previous data samples in memory, executing batch processing tasks, or using sliding windows. Therefore, it is very fast and computationally cheap. Hence, it is suitable for real-time applications.
- (iii) It is fully unsupervised, does not require offline training or prior knowledge about the data and can be started from an empty knowledge base.

#### 3.1. Data clouds

The granular structures used in AutoCloud are called data clouds [6]. Similarl to traditional clusters, they are local sets of data samples with common properties. Data clouds, however, do not have a particular shape or predefined boundaries. Therefore, they are a much more realistic representation of the actual data distribution than traditional fuzzy membership functions (e.g. triangular, trapezoidal, Gaussian), in which, instead, one only targets an approximation of an expected/desired distribution.

Although data clouds do not have specific shapes or boundaries (these are indirectly derived from the type of similarity measure used), they are visually represented as ellipses in this paper.

AutoCloud calculates the data eccentricity  $\zeta_k$  for each new data sample  $x_k$  in relation to each existing cloud independently, as shown in Fig. 1. In this example, note that AutoCloud determines the membership of the sample  $x_k$  in the two existing clouds  $c_1$  and  $c_2$ , based on the eccentricity  $\zeta_k$  for each cloud and the dynamic threshold proposed in Eq. (9). When this condition holds, AutoCloud determines that the membership of  $x_k$  to such cloud is significant enough that  $x_k$  affects that cloud.

Since AutoCloud is a recursive algorithm, past data samples do not to be stored in memory. Instead, only a small number of statistical variables are required for each cloud  $c_i$ : the number of samples ( $s_k^i$ ), the mean ( $\mu_k^i$ ) and the variance ( $[\sigma^2]_k^i$ ) of the samples that affect/belong to  $c_i$  at the time instant  $k$ .

For instance, Fig. 2 illustrates a scenario with two data clouds,  $c_1$  and  $c_2$ , after the reading of  $k$  samples. The number of samples that belong to each cloud is  $s_k^1 = 7$  and  $s_k^2 = 8$ , while three samples belong to both  $c_1$  and  $c_2$  simultaneously. Finally, the mean of each existing cloud,  $\mu_k^1$ ,  $\mu_k^2$ , respectively, are visually represented in the same image by the centres of  $c_1$  and  $c_2$ , while the variances  $[\sigma^2]_k^1$  and  $[\sigma^2]_k^2$  are represented by the spreading/radii of each cloud in the data space.

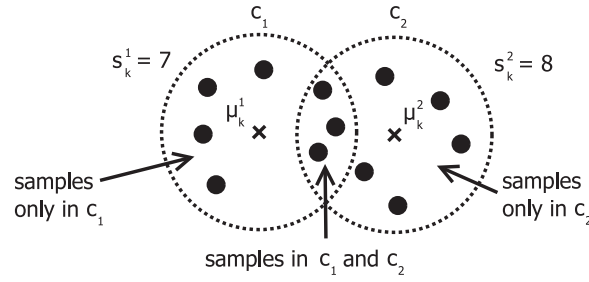


Fig. 2. Data clouds  $c_1$  and  $c_2$  at the time instant  $k$ .

### 3.2. Updating the data clouds

For each new data sample, AutoCloud determines its relevance to each existing cloud based on the value of the normalized eccentricity to  $c_i$ . When the eccentricity is significantly high, the sample  $x_k$  is considered *irrelevant* in relation to  $c_i$  and, therefore, does not affect that specific cloud. However, if  $x_k$  has low eccentricity, AutoCloud determines that the sample *belongs* to  $c_i$  (i.e.  $x_k$  is similar to the other samples that belong to  $c_i$ ) and updates that cloud. In that case, the number of samples  $s_k^i$ , the mean  $\mu_k^i$  and the variance  $[\sigma^2]^i_k$  are recursively updated to reflect the influence of the new sample on  $c_i$ .

It is important to highlight that a data sample can belong to multiple points simultaneously, preserving the nature of fuzzy membership. Alternatively, the algorithm may also determine that  $x_k$  has high eccentricity in relation to all existing clouds. In that case, a new cloud is created.

The threshold used to determine whether a sample  $x_k$  belongs or not to a cloud  $c_i$ , we generalize Eq. (9) to reflect the local membership in individual clusters:

$$\zeta^i(x_k) \leq \frac{m^2 + 1}{2s_k^i} \quad (10)$$

where  $m$  represents the sensitivity of the threshold. Hence, if the condition represented in Eq. (10) holds, it is determined that  $x_k$  belongs to  $c_i$ . On the other hand, if the condition does not hold,  $x_k$  does not belong to  $c_i$ .

The remaining equations are also generalized versions of the equations presented in Section 2. Thus, the eccentricity  $\xi^i(x_k)$  and the normalized eccentricity  $\zeta^i(x_k)$  of the sample  $x_k$  in relation to the  $i$ th data cloud are given by

$$\xi^i(x_k) = \frac{1}{[s_k^i]'} + \frac{([\mu_k^i]' - x_k)^T ([\mu_k^i]' - x_k)}{[s_k^i]' [\sigma^2]^i_k'} \quad (11)$$

$$\zeta^i(x_k) = \frac{\xi^i(x_k)}{2} \quad (12)$$

where  $[s_k^i]'$ ,  $[\mu_k^i]'$  and  $[\sigma^2]^i_k'$  are temporary values for the number of samples, mean and variance of the  $i$ th data cloud, respectively, supposing that  $x_k$  belongs to  $c_i$ . This supposition is necessary in order to verify, in sequence, whether  $x_k$  indeed belongs to  $c_i$ . These values are calculated by, respectively,

$$[s_k^i]' = s_{k-1}^i + 1 \quad (13)$$

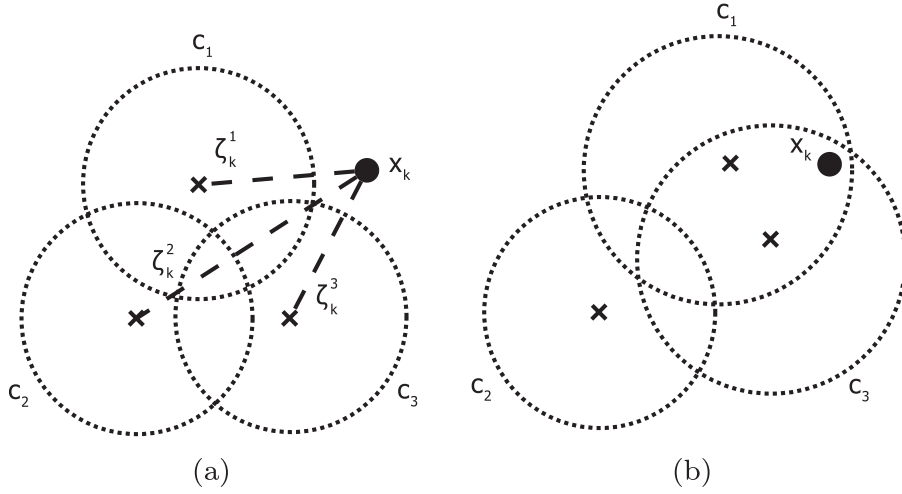
$$[\mu_k^i]' = \frac{[s_k^i]' - 1}{[s_k^i]'} \mu_{k-1}^i + \frac{1}{[s_k^i]'} x_k, \quad (14)$$

$$[\sigma^2]^i_k' = \frac{[s_k^i]' - 1}{[s_k^i]'} [\sigma^2]_{k-1}^i + \frac{1}{[s_k^i]'} \|x_k - [\mu_k^i]'\|^2 \quad (15)$$

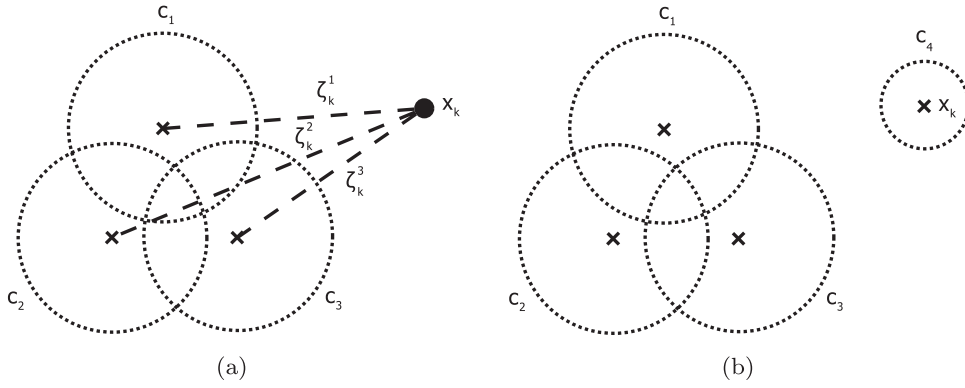
For each cloud  $c_i$ , with  $i = [1 \dots N]$ , where  $N$  is the number of existing clouds, and if Eq. (10) holds, the values of  $s_k^i$ ,  $\mu_k^i$  and  $[\sigma^2]^i_k$  are updated by, respectively,  $[s_k^i]'$ ,  $[\mu_k^i]'$  and  $[\sigma^2]^i_k'$ , previously calculated by Eqs. (13)–(15). This update must be executed since the supposition that  $x_k$  belongs to  $c_i$  is true and hence  $c_i$  needs to be updated to reflect the influence of  $x_k$ . On the other hand, if Eq. (10) does not hold, that means  $x_k$  does not belong and, therefore, should not affect  $c_i$ .

Fig. 3 illustrates this procedure by showing three data clouds ( $c_1$ ,  $c_2$  and  $c_3$ ) and an input sample  $x_k$ . According to the image, AutoCloud calculates the normalized eccentricity of  $x_k$  in relation to each cloud,  $\zeta^1(x_k)$ ,  $\zeta^2(x_k)$  and  $\zeta^3(x_k)$ , respectively. In this particular example, it can be seen that  $x_k$  belongs to both  $c_1$  and  $c_3$ , but not to  $c_2$ . Hence, only  $c_1$  and  $c_3$  are updated, while no action is performed for  $c_2$ , as shown in Fig. 3(b).

In the case where Eq. (10) does not hold for any of the existing  $N$  clouds, a new data cloud  $c_{N+1}$  is created and added to the rule basis. It is initialized with  $s_k^{N+1} = 1$ ,  $\mu_k^{N+1} = x_k$  and  $[\sigma^2]_k^{N+1} = 0$ . As an illustrative example, consider Fig. 4, in which the three clouds  $c_1$ ,  $c_2$  and  $c_3$  and a new data sample  $x_k$  are presented. It is easy to see in Fig. 4(a) that  $x_k$  is significantly distant from all existing data clouds, hence a new cloud  $c_4$  is created, as shown in Fig. 4(b).



**Fig. 3.** Updating of data clouds: (a)  $c_1$ ,  $c_2$ ,  $c_3$  and a newly read sample  $x_k$ , and (b)  $c_1$ ,  $c_2$ ,  $c_3$  after the update.



**Fig. 4.** Creation of a new data cloud: (a) three existing clouds,  $c_1$ ,  $c_2$ ,  $c_3$  and a new sample  $x_k$  and (b) a new cloud  $c_4$  is created.

### 3.3. Merging data clouds

In order to limit the number of clouds and, at the same time, preserve the evolving characteristics of this approach, AutoCloud is able to merge two partially overlapping data clouds, when appropriate. A merge is executed when the number of overlapping samples, i.e. the number of samples that belong to both clouds simultaneously, is significantly high. This operation is fully autonomous and non-parametric.

Given two clouds  $c_i$  and  $c_j$  at the time instant  $k$ , merge happens when at least one of the following conditions is true:

$$s_k^{c_i \cap c_j} > s_k^i - s_k^{c_i \cap c_j} \quad (16)$$

$$s_k^{c_i \cap c_j} > s_k^j - s_k^{c_i \cap c_j} \quad (17)$$

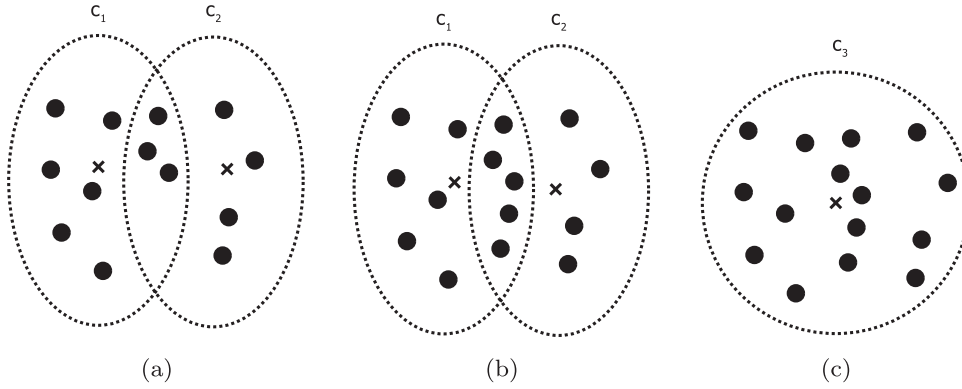
where  $s_k^{c_i \cap c_j}$  corresponds to the number of intersecting samples in  $c_i$  and  $c_j$  at the time instant  $k$ .

In summary, these two conditions tell us that two clouds are merged when the number of samples that belong to both clouds *simultaneously* is higher than the number of samples that belongs to only one of them *separately*. The process of merging overlapping data clouds prevents an uncontrolled growth in the rule/cloud basis.

Fig. 5 illustrates the process of merging clouds. In Fig. 5(a) the number of intersecting points in clouds  $c_1$  and  $c_2$  is less than the number of exclusive points in both clouds. Hence, in this case, no merge takes place. In contrast, Fig. 5(b) illustrates a concrete example of a merge, with the result cloud basis presented in Fig. 5(c).

When two clouds merge, the properties of the resulting cloud are defined by

$$s_k^l = s_k^i + s_k^j - s_k^{c_i \cap c_j} \quad (18)$$



**Fig. 5.** Merging of data clouds:  $c_1$  and  $c_2$ , (b)  $c_1$  and  $c_2$  right before merging and (c) resulting cloud  $c_3$  after merging.

$$\mu_k^l = \frac{s_k^i \mu_i + s_k^j \mu_j}{s_k^i + s_k^j} \quad (19)$$

$$[\sigma^2]_k^l = \frac{(s_k^i - 1)\sigma_i^2 + (s_k^j - 1)\sigma_j^2}{s_k^i + s_k^j - 2} \quad (20)$$

where the values of  $\mu_k^l$  and  $[\sigma^2]_k^l$  are obtained through a weighted mean of the respective values of each original cloud.

Analyzing the overall update, merge and creation of clouds, it is easy to see that AutoCloud is an algorithm that is able to handle two of the problems that are inherent to data stream analysis: concept drift and concept evolution.

### 3.4. The algorithm AutoCloud

Finally, [Algorithm 1](#) shows in detail each step of AutoCloud. In this example, we use the Euclidean distance as the underlying similarity measure between samples and clouds.

For each obtained data sample, AutoCloud determines if it belongs to any of the existing clouds. If so, any of the affected (and only the affected) clouds are updated in order to reflect the addition of this new sample. Otherwise, i.e. if the sample is significantly different from *all* existing clouds, a new cloud is created. Finally, each pair of clouds is analyzed in order to determine whether a merge is necessary. Taking  $n$  to be the number of features in the dataset and  $C_k$  the number of existing clusters at the time instant  $k$ , the complexity of AutoCloud is  $O(n \cdot C^2)$  for a data sample  $x_k$  processed at the time instant  $k$ .

Analyzing the algorithm, we can verify that at its startup the first data cloud is created and the first two samples obtained ( $k = 1$  and  $k = 2$ ) are added to this data cloud. The process of creating new data clouds starts only from the third sample. The reason for this is that the distance between the points is relative to the feature ranges. There is no way of determining whether two points are near or far if there is no pre-defined range, which is the case here. That is why a third point is expected before starting to determine how significant the distance between any two points is.

[Algorithms 2](#) and [3](#) detail the steps for the creation and merging of data clouds, respectively.

At the end of each iteration  $k$ , AutoCloud outputs the following values:

- The number of samples ( $s_k^i$ ), the mean ( $\mu_k^i$ ) and the variance ( $[\sigma^2]_k^i$ ) of each existing cloud ( $c_i$ ) at the time instant  $k$ .
- The list with membership degrees ( $\gamma_{c_i}$ ) of the sample  $x_k$  in each cloud ( $c_i$ ).

## 4. Results

To validate AutoCloud, we will present in this section the results obtained by applying the proposed algorithm to data stream clustering problems. These results were obtained using data sets already consolidated and widely used in the field of machine learning. In addition to this data, we also used some artificially generated data to demonstrate some features of AutoCloud.

### 4.1. Data sets

The data sets used were obtained from a specific repository for data clustering [\[31\]](#). This repository has some data sets, both artificial and real, that are widely used in data clustering tasks. For each data set, the number of samples and the number of existing clusters are specified. In some cases, the centroid of each group is also available as well as the information about to which group each sample belongs.

**Algorithm 1:** Algorithm *Auto-Cloud* using Euclidean distance.

---

**Input:** set of samples  $X = \{x_1, x_2, x_3, \dots\} \in R^n$

```

1 begin
2   while  $x_k \leftarrow \text{read next sample}$  do
3     if  $k = 1$  then
4       // create data cloud 1 and add  $x_1$ 
5        $s_1^1 \leftarrow 1$ 
6        $\mu_1^1 \leftarrow x_1$ 
7        $[\sigma^2]_1^1 \leftarrow 0$ 
8        $N \leftarrow 1$ 
9     else
10      if  $k = 2$  then
11        // add  $x_2$  in data cloud 1
12         $s_2^1 \leftarrow 2$ 
13         $\mu_2^1 \leftarrow (\mu_1^1 + x_2)/2$ 
14         $[\sigma^2]_2^1 \leftarrow \|x_2 - \mu_2^1\|/2$ 
15      else
16        if  $k \geq 3$  then
17          forall the existing data cloud  $c_i$  do
18            calculate  $[s']_k^i$ ,  $[\mu']_k^i$  and  $[[\sigma^2]']_k^i$  by equations 13, 14 e 15
19            calculate  $\zeta_k^i$  by equation 12
20            if  $\zeta_k^i \leq (m^2 + 1)/(2 * [s']_k^i)$  then
21              // add  $x_k$  in data cloud  $i$ 
22               $s_k^i \leftarrow [s']_k^i$ 
23               $\mu_k^i \leftarrow [\mu']_k^i$ 
24               $[\sigma^2]_k^i \leftarrow [[\sigma^2]']_k^i$ 
25            else
26               $s_k^i \leftarrow s_{k-1}^i$ 
27               $\mu_k^i \leftarrow \mu_{k-1}^i$ 
28               $[\sigma^2]_k^i \leftarrow [\sigma^2]_{k-1}^i$ 
29            end
30          end
31          if  $x_k \notin c_i, \forall i$  then
32            createDataCloud( $x_k$ )
33          end
34          forall the pair of data clouds  $c_i$  and  $c_j$  do
35            verifyMerge( $c_i, c_j$ )
36          end
37        end
38      end
39    end
40  end

```

---

**Algorithm 2:** Procedure that creates a new data cloud using Euclidean distance.

---

```

1 procedure createDataCloud ( $x_k$ )
2   // create new data cloud  $l$ 
3    $n_k^l \leftarrow 1$ 
4    $\mu_k^l \leftarrow x_k$ 
5    $[\sigma^2]_k^l \leftarrow 0$ 
6    $\alpha_{c_l} \leftarrow 1$ 
7    $N \leftarrow N + 1$ 
8 end

```

---



---

**Algorithm 3:** Procedure that performs the merge between two data clouds using Euclidean distance.

---

```

1 procedure verifyMerge ( $c_i, c_j$ )
2   if  $s_k^{c_i \cap c_j} > s_k^i - s_k^{c_i \cap c_j}$  or  $s_k^{c_i \cap c_j} > s_k^j - s_k^{c_i \cap c_j}$  then
3     // merge data clouds  $i$  and  $j$  to data cloud  $l$ 
4     calculate  $s_k^l, \mu_k^l$  and  $[\sigma^2]_k^l$  by Eqs. (18)–(20)
5      $N \leftarrow N - 1$ 
6   end
7 end

```

---

**Table 1**

Synthetic data sets used in data clustering experiments.

| Data set    | Category        | N    | K  | Dimension |
|-------------|-----------------|------|----|-----------|
| S1          | S-sets          | 5000 | 15 | 2         |
| S2          |                 | 5000 | 15 | 2         |
| A1          |                 | 3000 | 20 | 2         |
| A2          |                 | 5250 | 35 | 2         |
| dim512      | Dim-sets (high) | 1024 | 16 | 512       |
| dim1024     |                 | 1024 | 16 | 1024      |
| Aggregation | Shape-sets      | 788  | 7  | 2         |
| Compound    |                 | 399  | 6  | 2         |
| Unbalance   | Unbalance       | 6500 | 8  | 2         |

**Table 2**

Results obtained in the clustering of analyzed data sets.

| Data set    | K  | $\hat{K}$ | Quality | $t_s$ (s) | $t_a$ (ms) |
|-------------|----|-----------|---------|-----------|------------|
| S1          | 15 | 15        | 0.30    | 3.71      | 0.74       |
| S2          | 15 | 15        | 0.41    | 3.48      | 0.70       |
| A1          | 20 | 20        | 0.30    | 3.0       | 1.00       |
| A2          | 35 | 35        | 0.23    | 11.79     | 2.24       |
| dim512      | 16 | 16        | 0.16    | 22.52     | 2.20       |
| dim1024     | 16 | 16        | 0.39    | 90.13     | 88.02      |
| Aggregation | 7  | 7         | 0.52    | 0.18      | 0.23       |
| Compound    | 6  | 4         | 0.53    | 0.07      | 0.10       |
| Unbalance   | 8  | 8         | 0.34    | 1.21      | 0.19       |

The data sets of the adopted repository are divided into several categories. Among the existing synthetic data categories, we have made use of the following: S-sets, A-sets, DIM-sets (high), Shape-sets, and Unbalance.

We have selected some of the data sets belonging to these categories to perform our experiments. Table 1 shows a summary of the selected data sets, indicating the number of samples,  $N$ , the number of clusters,  $K$ , and the size of each cluster. Note that most of the selected data is 2-dimensional, but this is not a prerequisite for AutoCloud, since it is capable of handling data of any dimensionality. This choice was made only to facilitate the visualization of the data and the obtained results. To prove this, we will also use two high-dimensional data sets, dim512 and dim1024, which have dimensions 512 and 1024, respectively.

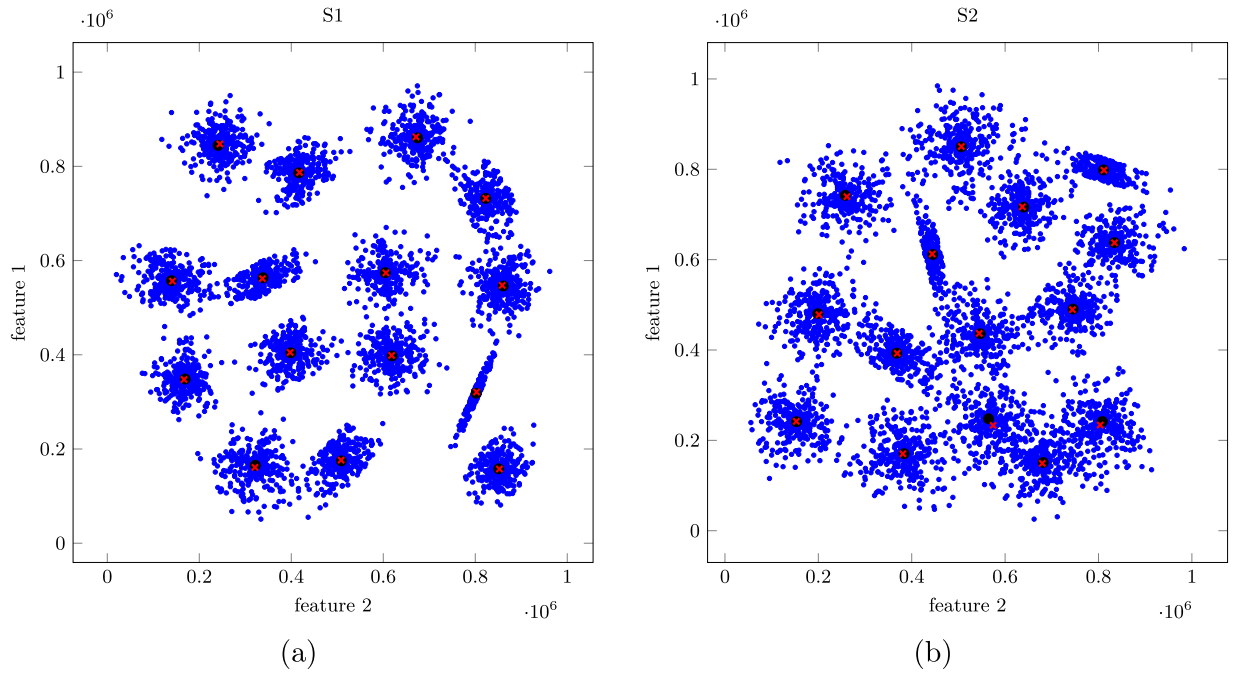
#### 4.2. Experimental results

AutoCloud was applied to each of the selected data sets in order to evaluate the proposed clustering approach. Table 2 shows the number of data clouds obtained,  $\hat{K}$ , the quality of the data clouds obtained, the processing time for each data stream,  $t_s$ , and the average processing time of each sample,  $t_a$ . This table also shows the real number of clusters,  $K$ , in each data set.

Analyzing the results shown in Table 2, we can verify that AutoCloud was able to accurately identify the actual number of clusters in almost all analyzed data sets. The number of data clouds created by AutoCloud was not equal to the number of clusters in the compound base, where AutoCloud created only 4 data clouds, two less than expected. In this experiment, the performance of AutoCloud was satisfactory, regardless of the dimensionality of the data sets analyzed, since it was able to correctly identify the number of clusters in the low- and high-dimensional data sets.

To measure the quality of the obtained data clouds, we used the same metric used by Lughofer and Sayed-Mouchaweh [17]. According to this metric, the quality value is calculated using the following equation:

$$\text{quality} = \frac{\sum_{i=1}^C \sum_{j=1}^N \mu_{ij}^2 ||\bar{x}_i - \bar{x}_j||}{N \min_{i,j=1,\dots,C, i \neq j} (||\bar{c}_i - \bar{c}_j||)} \quad (21)$$



**Fig. 6.** Obtained centroids in clustering of S-Sets: (a) S1 and (b) S2.

where  $C$  denotes the number of data clouds obtained,  $N$  is the total number of samples, and  $\mu_{ij}$  is the membership degree of sample  $j$  in data cloud  $i$ . The value of  $\mu_{ij}$  was calculated using the typicality of sample  $j$  in data cloud  $i$ . The quality values obtained were similar to those obtained by Lughofer and Sayed-Mouchaweh [17] using AutoClust on data sets A1, A2, S1 and S2. The lower the calculated value, the better the quality of the data clouds.

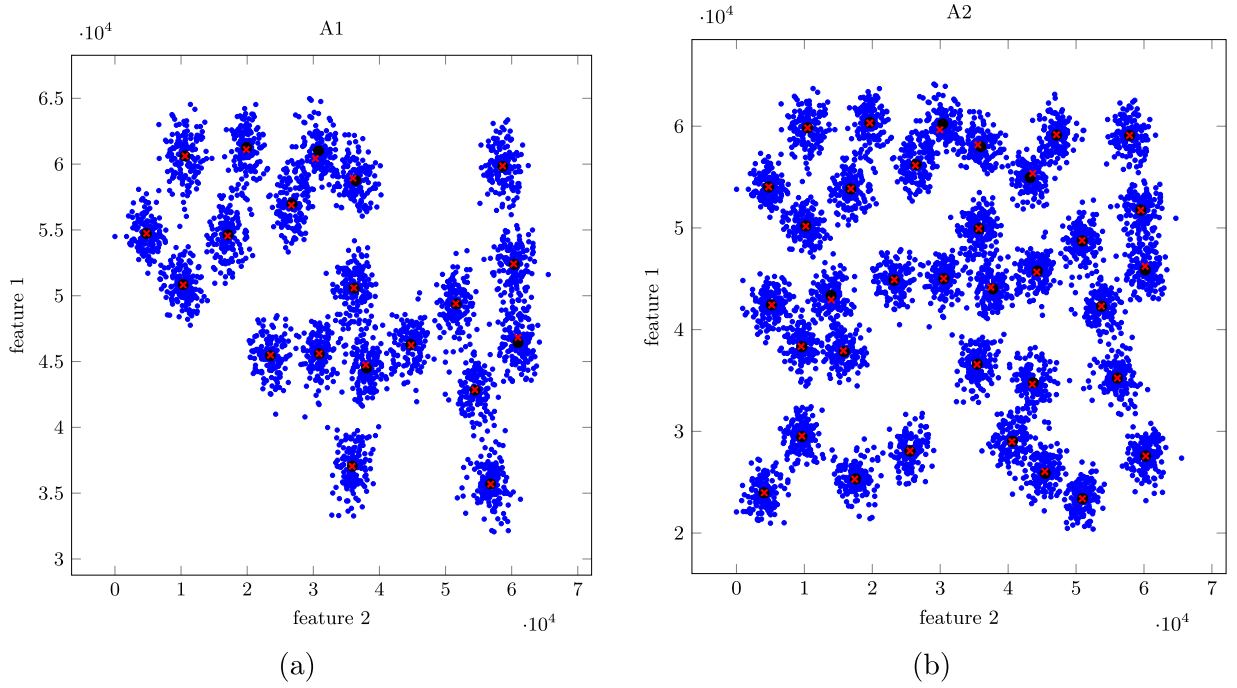
Regarding the processing time required by AutoCloud, we verified that this value is proportional to the size of the analyzed data stream, as well as the dimension of the samples. The largest obtained values were in the processing of the two high-dimensional data sets used (dim512 and dim1024). It is important to note that the measured processing time does not take into account any sampling rate when reading the data. The average processing time of each sample was obtained by dividing the total processing time of the stream by the number of samples in that stream. Analyzing the obtained values of  $t_a$ , AutoCloud proved to be a time-efficient algorithm and adequate for on-line applications. In the worst presented case, each sample was processed in about 88 ms. This result allows AutoCloud to be used in time-constrained applications that require fast responses.

In order to evaluate the quality of the obtained data clouds in the experiments, we have to go beyond the identified number of clusters/data clouds. It is also necessary to verify that the centroid of each identified data cloud was calculated correctly by comparing the obtained with the expected centroids. The obtained centroids in each of the analyzed data sets are shown in Figs. 6–10. In each of these figures the samples belonging to each data set are shown in blue. The black dots represent the real centroids of the clusters and the red dots correspond to the centroids of the data clouds obtained by AutoCloud.

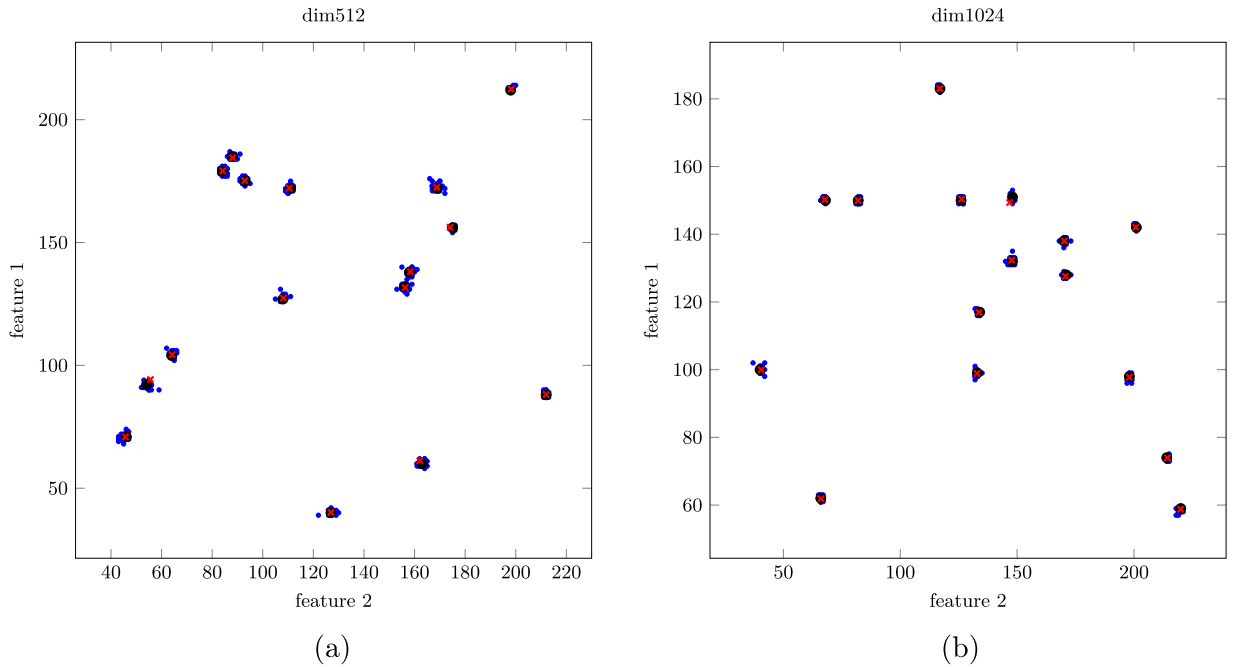
The centroids obtained for data sets S1 and S2 are shown in Fig. 6. In the image, it is possible to see that the data are grouped in well-defined clusters, but with different shapes, numbers of samples and spreading of samples. In data set S2, the clusters are closer to each other than in S1, thus there is a greater degree of overlap between them in that set. It is also possible to see that the centroids of all the data clouds obtained correspond to the expected values in both S1 and S2. This can be verified by observing that the points marked with red X coincide with the points marked with the black circle.

The obtained centroids in sets A1 and A2 are shown in Fig. 7. We have verified that the clusters are well defined in these two sets. Another characteristic of these two sets is that the number of samples in each of the clusters is equal. By observing the figure, we find that the centroids obtained in all data clouds correspond to the real values expected for each cluster in the two sets.

Fig. 8 shows the obtained centroids in the high-dimensional sets dim512 and dim1024. The data from these two sets were presented to AutoCloud in its entirety, without any prior feature selection. However, to facilitate the visualization of the data and the results obtained, the graphs presented in this figure show only the first two dimensions of the samples. Once again, the centroids in each identified data cloud correspond to the expected values, regardless of the high dimensionality of the analyzed data. In only a few cases, one in the set dim512 and two in the set dim1024, was there a small error in the identified centroids. However, this error was not very significant, still allowing the correct identification of data clouds.

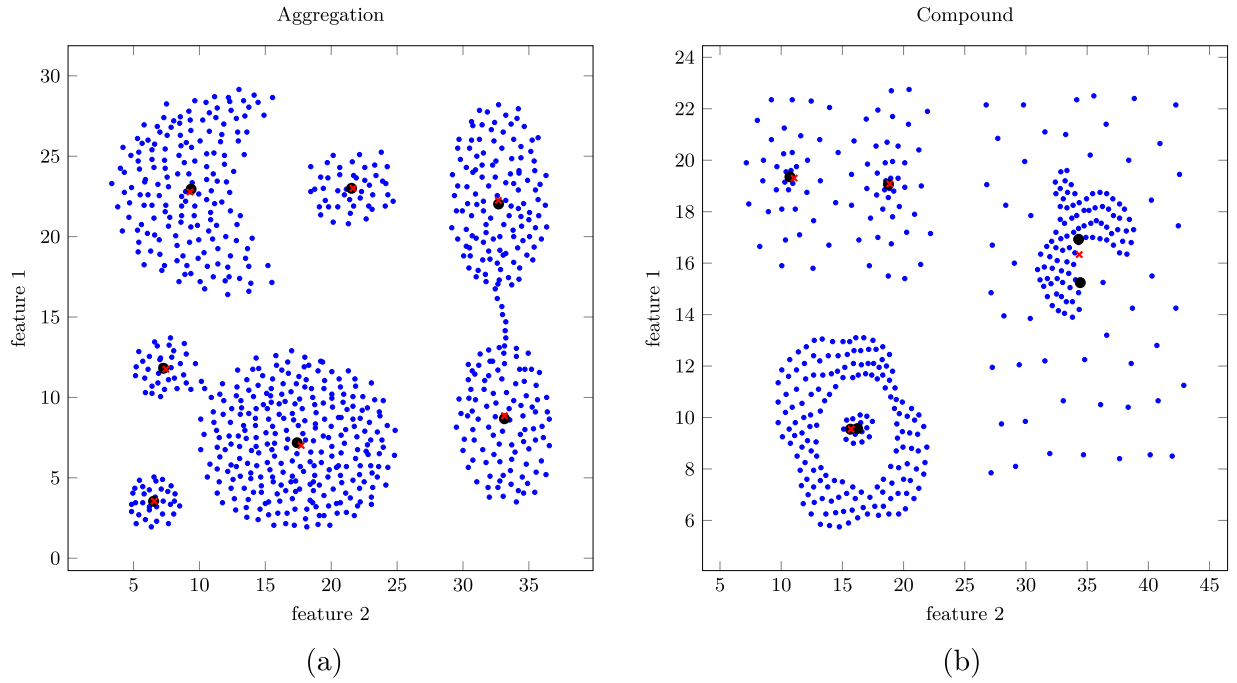


**Fig. 7.** Obtained centroids in clustering of A-Sets: (a) A1 and (b) A2.

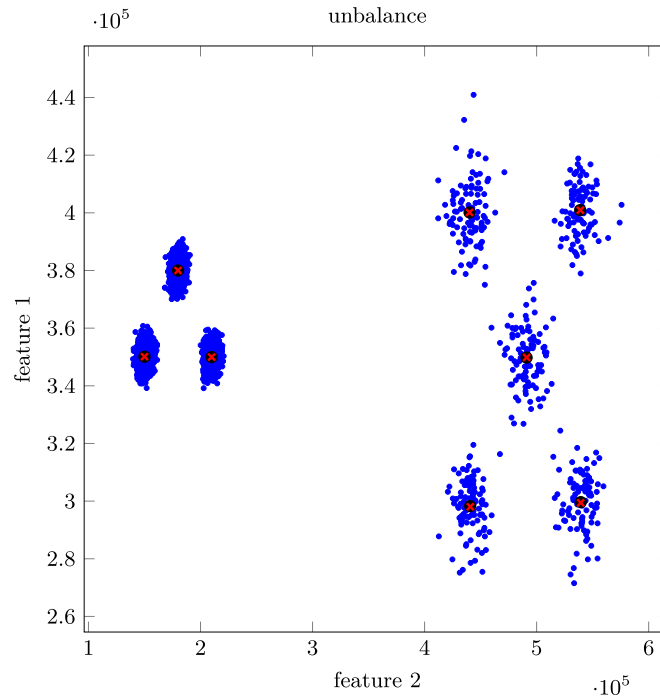


**Fig. 8.** Obtained centroids in clustering of Dim-Sets: (a) dim512 and (b) dim1024.

The obtained results for the Shape-sets used in the experiments are shown in Fig. 9. These two data sets are the ones with the fewest samples among those analyzed. In addition, the clusters have different shapes and sizes, but can also be easily identified visually. In the Aggregation set, all centroids were identified quite accurately by AutoCloud. In the Compound set, an error occurred in the identification of the clusters. In this data set the number of identified data clouds did not match the number of expected clusters. Analyzing Fig. 9(b) we find that there are two clusters that are inside two other



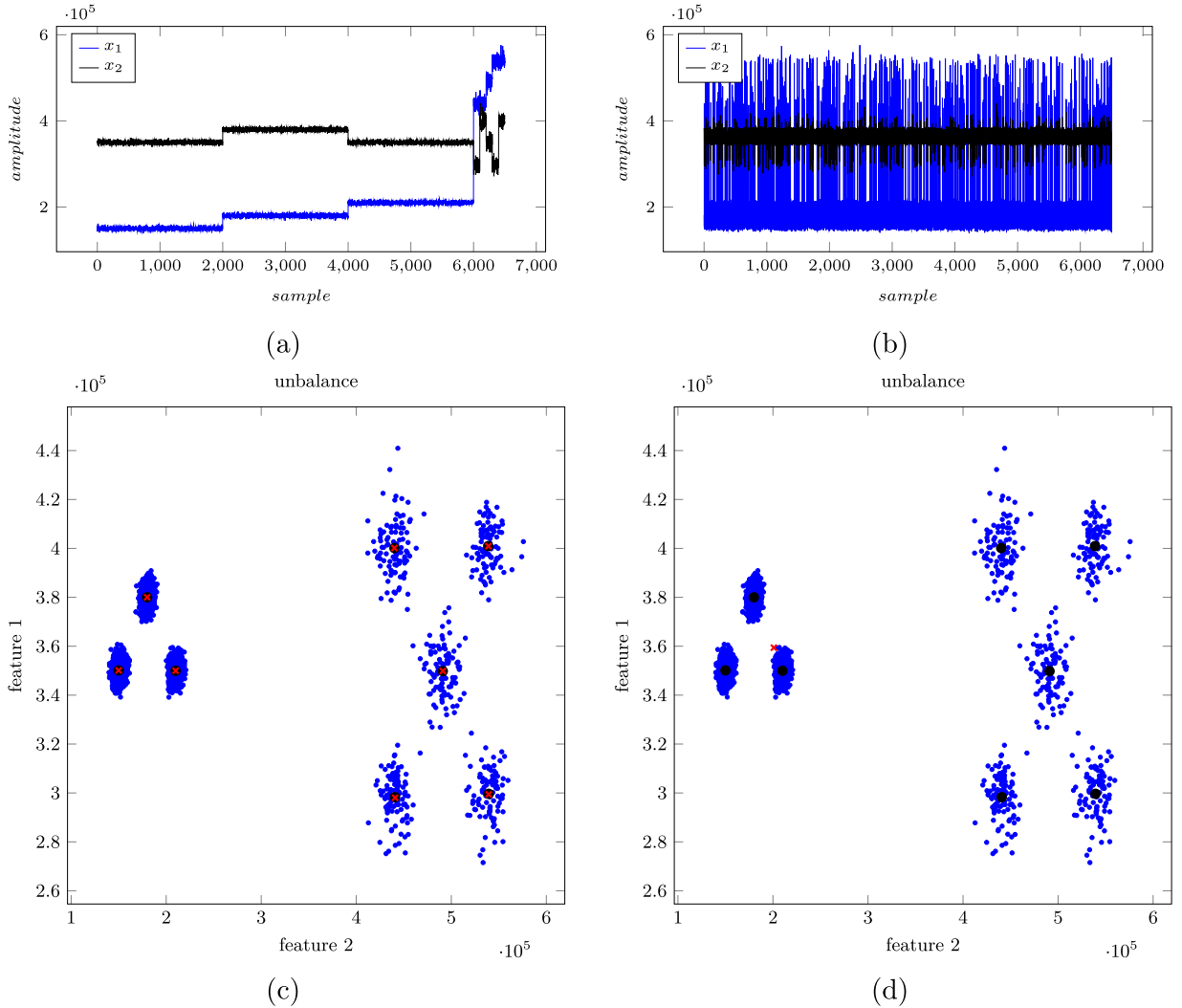
**Fig. 9.** Obtained centroids in clustering of Shape-Sets: (a) Aggregation and (b) Compound.



**Fig. 10.** Obtained centroids in clustering of Unbalance.

clusters. This caused each of these two pairs of clusters to be identified as a single data cloud. This caused the number of data clouds identified to be two units smaller than expected.

Finally, Fig. 10 shows the centroids obtained for the data set Unbalance. This data set presents 8 clusters and most samples are concentrated in 3 of them. These 3 clusters have 2000 samples each, while the remaining 5 have only 100 samples each. We can observe in Fig. 10 that the centroids were again obtained satisfactorily, regardless of the number of samples presented by each cluster.



**Fig. 11.** Obtained centroids in clustering of data set *Unbalance* modified: (a) original data stream, (b) randomized data stream, (c) obtained centroids for original data stream and (d) obtained centroids for randomized data stream.

Based on the presented results, it is possible to note that AutoCloud was able to determine the *clusters* and their respective centres adequately. It is important to highlight that all input data were processed in the form of data streams by AutoCloud. More specifically, at the  $k$ th time instant, AutoCloud only had access to the  $k$ th data sample. In addition, AutoCloud does not require storing previous data samples.

Aiming to analyze the influence of the order of samples in the data stream on the clustering in an evolving context, we used again the *unbalance* data set, but the order of the data was randomized. Fig. 11 shows the obtained results with this experiment. Fig. 11(a) presents the original data stream and Fig. 11(b) presents the randomized data stream. Fig. 11(c) and Fig. 11(d) present the obtained clusters by AutoCloud for the original data stream and the randomized data stream, respectively. The obtained centroids are indicated by red dots. As expected, for the former case the algorithm obtained 8 clusters, while for the latter it resulted in a single cluster, since the nature of the algorithm requires the samples to be processed in the order they are acquired.

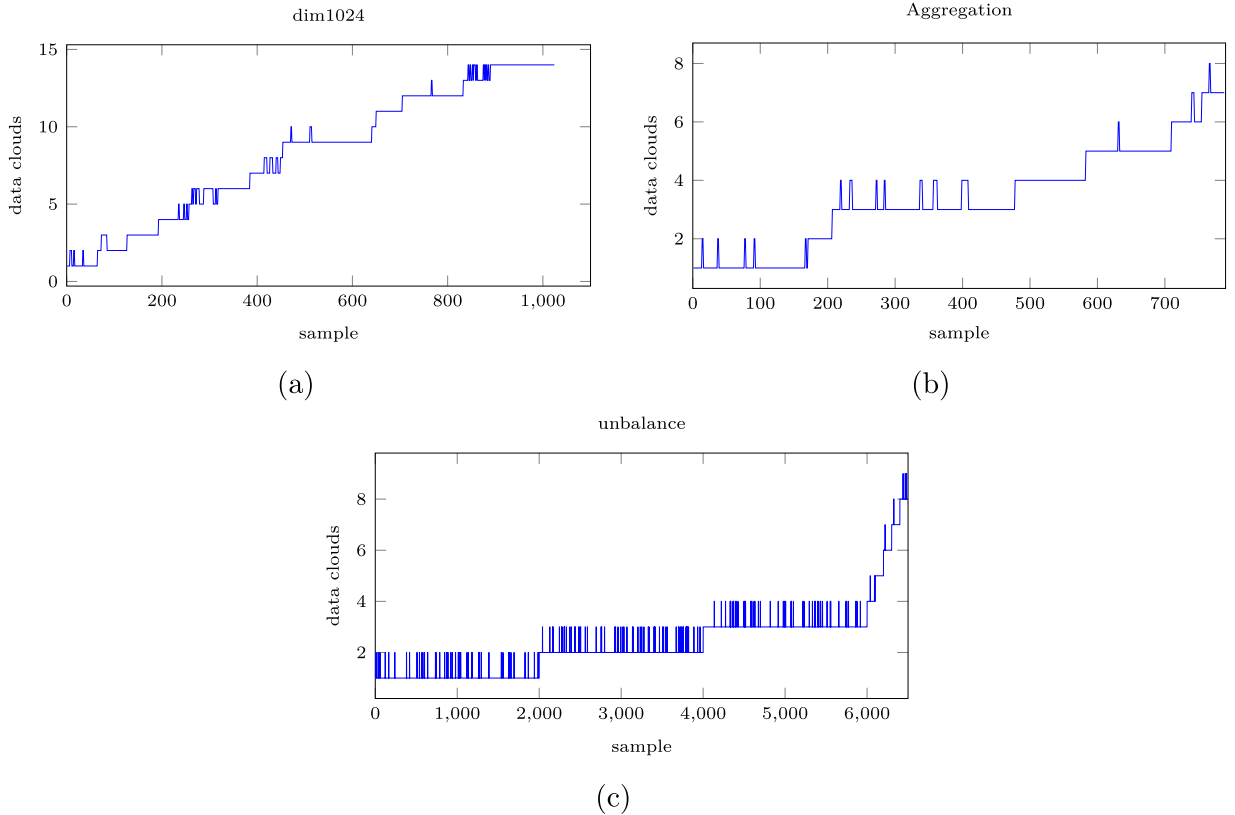
Therefore, it is important to highlight that AutoCloud is not suitable for the analysis of data that does not have a time dependency between the samples. However, for the vast majority of real-world problems, one of the main characteristics of data streams is that they carry very strong temporal relationships between neighbouring data samples (e.g. in a control process, none of the input/output variables are expected to randomly oscillate between two sequential time instants). That is precisely the type of problem AutoCloud has been proposed to address.

The only tunable parameter of AutoCloud is the value of  $m$ , which is used to determine the threshold defined by Eq. (9). All previous presented results used  $m = 2$ , which is equivalent to using a  $2\sigma$  threshold under normal distribution. This parameter was set based on previous successful work [8,30]. Then, in order to analyze the impact of the parameter  $m$  on

**Table 3**

Obtained results of analyzed data sets using several values of  $m$ .

| Data set    | K  | $\hat{K}_m = 1$ | $\hat{K}_m = 2$ | $\hat{K}_m = 3$ |
|-------------|----|-----------------|-----------------|-----------------|
| S1          | 15 | 114             | 15              | 10              |
| S2          | 15 | 123             | 15              | 1               |
| A1          | 20 | 150             | 20              | 1               |
| A2          | 35 | 250             | 35              | 1               |
| dim512      | 16 | 30              | 16              | 16              |
| dim1024     | 16 | 42              | 16              | 16              |
| Aggregation | 7  | 278             | 7               | 1               |
| Compound    | 6  | 126             | 4               | 1               |
| Unbalance   | 8  | 103             | 8               | 8               |

**Fig. 12.** Creation and merging of data clouds in data sets (a) dim1024, (b) Aggregation and (c) Unbalance.

AutoCloud's performance, we have used  $m = 1, 2$  and  $3$  on the data sets used previously. The obtained results are presented in Table 3 and confirm that AutoCloud's sensitivity for creating new clusters increases when the value of  $m$  decreases and vice versa.

Although we have used only integral values for  $m$ , any real positive number can be used. Thus, by adjusting this value, AutoCloud is able to achieve different degrees of granularization during the identification of the data clouds. The best value of  $m$  will depend on the characteristics of the analyzed data distribution. However, once AutoCloud is executed online from the very first data sample,  $m$  can only be optimized if previous information about the data is available, which is not the case for the problems presented in this paper. As future research, we will investigate how to relate the value of  $m$  to some measure of the dispersion of the data. This would allow estimating a value of  $m$  adaptively, updating it for each sample analyzed.

AutoCloud is built to cope with concept evolution by design, since it starts from scratch and creates and updates clusters on-demand. In Fig. 12, we illustrate the processes of creation and merging of data clouds for the dim1024, aggregation, and unbalance data sets.

It is possible to observe that as the data stream is processed, the number of data clouds tends to increase. This is due to the fact that AutoCloud can identify new concepts in the data stream and create new clusters to represent them properly.

On the other hand, merges of data clouds tend to occur soon after the creation of a new cluster since, in many cases, the algorithm realizes that the creation of the data cloud was not necessary, and thus performs a self-regulating activity by the merging procedure.

## 5. Conclusion

In this paper we presented an algorithm, called AutoCloud, for data stream clustering. The obtained clusters are called data clouds. AutoCloud is an evolving algorithm capable of autonomously identifying changes that occur over time in the distribution of the data stream. Since it is evolving, it can update the parameters of the data cloud (concept drift), create new data clouds (concept evolution), and merge existing data clouds, in order to auto-adapt to changes in the data stream over time.

Based on the presented results, AutoCloud was able to correctly identify both the number and the location of the analyzed clusters. In addition, the proposed strategy for merging the data clouds proved to be quite efficient. Another relevant aspect that must be highlighted is that AutoCloud does not use representation models of the distribution of the data, thus it was able to identify clusters regardless of their format.

Since AutoCloud is executed recursively and does not require data samples to be stored, it is very computationally efficient, processing- and memory-wise.

For future research, in order to improve AutoCloud, we intend to investigate the use of the Mahalanobis distance as a metric and to propose efficient strategies to split clouds dynamically. In addition, we will also investigate how to obtain an adaptive value of  $m$ , with this value being updated with each new sample analyzed.

## Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## CRedit authorship contribution statement

**Clauber Gomes Bezerra:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing - original draft, Writing - review & editing, Visualization. **Bruno Sielly Jales Costa:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing - original draft, Writing - review & editing, Visualization, Project administration, Resources. **Luiz Affonso Guedes:** Conceptualization, Methodology, Formal analysis, Investigation, Writing - original draft, Writing - review & editing, Visualization, Project administration, Resources, Supervision. **Plamen Parvanov Angelov:** Conceptualization, Resources.

## References

- [1] D. Xu, Y. Tian, A comprehensive survey of clustering algorithms, *Ann. Data Sci.* 2 (2) (2015) 165–193, doi:[10.1007/s40745-015-0040-1](https://doi.org/10.1007/s40745-015-0040-1).
- [2] A. Saxena, M. Prasad, A. Gupta, N. Bharill, O.P. Patel, A. Tiwari, M.J. Er, W. Ding, C.-T. Lin, A review of clustering techniques and developments, *Neurocomputing* 267 (2017) 664–681, doi:[10.1016/j.neucom.2017.06.053](https://doi.org/10.1016/j.neucom.2017.06.053).
- [3] C.C. Aggarwal, C.K. Reddy, *Data Clustering: Algorithms and Applications*, first ed., Chapman & Hall/CRC, 2013.
- [4] J. Macqueen, Some methods for classification and analysis of multivariate observations, in: *5th Berkeley Symposium on Mathematical Statistics and Probability*, 1967, pp. 281–297.
- [5] J.A. Silva, E.R. Faria, R.C. Barros, E.R. Hruschka, A.C.P.L.F. de Carvalho, J. Gama, Data stream clustering: a survey, *ACM Comput. Surv.* 46 (1) (2013) 13:1–13:31, doi:[10.1145/2522968.2522981](https://doi.org/10.1145/2522968.2522981).
- [6] P. Angelov, *Autonomous Learning Systems: From Data to Knowledge in Real Time*, John Wiley and Sons, 2012.
- [7] F. Gomide, E. Lughofer, Recent advances on evolving intelligent systems and applications, *Evol. Syst.* 5 (4) (2014) 217–218, doi:[10.1007/s12530-014-9121-1](https://doi.org/10.1007/s12530-014-9121-1).
- [8] I. Škrjanc, J.A. Iglesias, A. Sanchis, D. Leite, E. Lughofer, F. Gomide, Evolving fuzzy and neuro-fuzzy approaches in clustering, regression, identification, and classification: a survey, *Inf. Sci.* 490 (2019) 344–368.
- [9] P. Angelov, D.P. Filev, N. Kasabov, *Evolving Intelligent Systems: Methodology and Applications*, Wiley-IEEE Press, 2010.
- [10] P. Angelov, X. Zhou, Evolving fuzzy-rule-based classifiers from data streams, *IEEE Trans. Fuzzy Syst.* 16 (6) (2008) 1462–1475, doi:[10.1109/TFUZZ.2008.925904](https://doi.org/10.1109/TFUZZ.2008.925904).
- [11] J. Iglesias, A. Ledezma, A. Sanchis, P. Angelov, Real-time recognition of calling pattern and behaviour of mobile phone users through anomaly detection and dynamically evolving clustering, *Appl. Sci.* 7 (8) (2017) 1–14, doi:[10.3390/app7080798](https://doi.org/10.3390/app7080798).
- [12] B.S.J. Costa, P. Angelov, L.A. Guedes, Fully unsupervised fault detection and identification based on recursive density estimation and self-evolving cloud-based classifier, *Neurocomputing* 150 (A) (2015) 289–303, doi:[10.1016/j.neucom.2014.05.086](https://doi.org/10.1016/j.neucom.2014.05.086).
- [13] D. Kangin, P. Angelov, J.A. Iglesias, Autonomously evolving classifier {TEDAClass}, *Inf. Sci.* 366 (2016) 1–11 <https://doi.org/10.1016/j.ins.2016.05.012>.
- [14] M. Traore, A. Chammas, E. Duviella, Supervision and prognosis architecture based on dynamical classification method for the predictive maintenance of dynamical evolving systems, *Reliab. Eng. Syst. Saf.* 136 (2015) 120–131, doi:[10.1016/j.ress.2014.12.005](https://doi.org/10.1016/j.ress.2014.12.005).
- [15] M. Pratama, J. Lu, E. Lughofer, G. Zhang, S. Anavatti, Scaffolding type-2 classifier for incremental learning under concept drifts, *Neurocomput.* 191 (C) (2016) 304–329, doi:[10.1016/j.neucom.2016.01.049](https://doi.org/10.1016/j.neucom.2016.01.049).
- [16] D. Leite, P. Costa, F. Gomide, Evolving granular neural networks from fuzzy data streams, *Neural Netw.* 38 (2013) 1–16, doi:[10.1016/j.neunet.2012.10.006](https://doi.org/10.1016/j.neunet.2012.10.006).
- [17] E. Lughofer, M. Sayed-Mouchaweh, Autonomous data stream clustering implementing split-and-merge concepts - towards a plug-and-play approach, *Inf. Sci.* 304 (C) (2015) 54–79, doi:[10.1016/j.ins.2015.01.010](https://doi.org/10.1016/j.ins.2015.01.010).
- [18] E. Lughofer, Extensions of vector quantization for incremental clustering, *Pattern Recognit.* 41 (3) (2008) 995–1011, doi:[10.1016/j.patcog.2007.07.019](https://doi.org/10.1016/j.patcog.2007.07.019).
- [19] A. Lemos, W. Caminhas, F. Gomide, Adaptive fault detection and diagnosis using an evolving fuzzy classifier, *Inf. Sci.* 220 (2013) 64–85, doi:[10.1016/j.ins.2011.08.030](https://doi.org/10.1016/j.ins.2011.08.030).

- [20] P. Angelov, Anomaly detection based on eccentricity analysis, in: *Proc. IEEE Symposium Series in Computational Intelligence (SSCI 2014)*, Orlando, Florida, U.S.A., 2014, pp. 1–8.
- [21] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, in: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining, KDD'96*, AAAI Press, 1996, pp. 226–231.
- [22] J. Shao, Y. Tan, L. Gao, Q. Yang, C. Plant, I. Assent, Synchronization-based clustering on evolving data stream, *Inf. Sci.* 501 (2019) 573–587, doi:10.1016/j.ins.2018.09.035.
- [23] P. Kranen, I. Assent, C. Baldauf, T. Seidl, The clustree: indexing micro-clusters for anytime stream mining, *Knowl. Inf. Syst.* 29 (2) (2011) 249–272, doi:10.1007/s10115-010-0342-8.
- [24] C. Isaksson, M.H. Dunham, M. Hahsler, SOStream: self organizing density-based clustering over data stream, in: *Proceedings of the 8th International Conference on Machine Learning and Data Mining in Pattern Recognition*, Springer-Verlag, Berlin, 2012, pp. 264–278, doi:10.1007/978-3-642-31537-4\_21.
- [25] R. Hyde, P. Angelov, A. Mackenzie, Fully online clustering of evolving data streams into arbitrarily shaped clusters, *Inf. Sci.* 382 (2016) 1–41, doi:10.1016/j.ins.2016.12.004.
- [26] M.K. Islam, A buffer-based online clustering for evolving data stream, *Inf. Sci.* 489 (2019) 113–135.
- [27] D. Kangin, P. Angelov, Evolving clustering, classification and regression with TEDA, in: *Proceedings of the 2015 International Joint Conference on Neural Networks, IEEE, Killarney, Ireland, 2015*, pp. 1–8.
- [28] B.S.J. Costa, C.G. Bezerra, L.A. Guedes, P.P. Angelov, Online fault detection based on typicality and eccentricity data analytics, in: *Proc. 2015 International Joint Conference on Neural Networks, IEEE, Killarney, Ireland, 2015*, pp. 1–6, doi:10.1109/IJCNN.2015.7280712.
- [29] J.G. Saw, M. Yang, T.C. Mo, Chebyshev inequality with estimated mean and variance, *Am. Stat.* 38 (2) (1984) 130–132.
- [30] G.E. Cook, J.E. Maxwell, R.J. Barnett, A.M. Strauss, Statistical process control application to weld process, *IEEE Trans. Ind. Appl.* 33 (2) (1997) 454–463, doi:10.1109/28.568010.
- [31] Clustering datasets - Joensuu, 2015, (<https://cs.joensuu.fi/sipu/datasets/>). Accessed on 27/1/2017.